

cars-price-prediction-model

February 3, 2026

```
[80]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
```

```
[81]: df = pd.read_csv("car - prediction.csv")
```

```
[82]: df.head()
```

```
[82]:
```

	name	year	selling_price	km_driven	fuel	\
0	Maruti 800 AC	2007	60000	70000	Petrol	
1	Maruti Wagon R LXI Minor	2007	135000	50000	Petrol	
2	Hyundai Verna 1.6 SX	2012	600000	100000	Diesel	
3	Datsun RediGO T Option	2017	250000	46000	Petrol	
4	Honda Amaze VX i-DTEC	2014	450000	141000	Diesel	

	seller_type	transmission	owner
0	Individual	Manual	First Owner
1	Individual	Manual	First Owner
2	Individual	Manual	First Owner
3	Individual	Manual	First Owner
4	Individual	Manual	Second Owner

0.0.1 What this does

- `frac=1` → take 100% of rows
- `random_state=42` → reproducible shuffle
- `reset_index(drop=True)` → clean index

```
[83]: X = df.drop(columns=["selling_price"], axis=1)
X
```

```
[83]:
```

	name	year	km_driven	fuel	\
0	Maruti 800 AC	2007	70000	Petrol	
1	Maruti Wagon R LXI Minor	2007	50000	Petrol	
2	Hyundai Verna 1.6 SX	2012	100000	Diesel	

3	Datsun RediGO T Option	2017	46000	Petrol
4	Honda Amaze VX i-DTEC	2014	141000	Diesel
...	...	...	...	...
4335	Hyundai i20 Magna 1.4 CRDi (Diesel)	2014	80000	Diesel
4336	Hyundai i20 Magna 1.4 CRDi	2014	80000	Diesel
4337	Maruti 800 AC BSIII	2009	83000	Petrol
4338	Hyundai Creta 1.6 CRDi SX Option	2016	90000	Diesel
4339	Renault KWID RXT	2016	40000	Petrol

	seller_type	transmission	owner
0	Individual	Manual	First Owner
1	Individual	Manual	First Owner
2	Individual	Manual	First Owner
3	Individual	Manual	First Owner
4	Individual	Manual	Second Owner
...	...	...	...
4335	Individual	Manual	Second Owner
4336	Individual	Manual	Second Owner
4337	Individual	Manual	Second Owner
4338	Individual	Manual	First Owner
4339	Individual	Manual	First Owner

[4340 rows x 7 columns]

```
[84]: y = df["selling_price"]
      y
```

```
[84]: 0      60000
      1     135000
      2     600000
      3     250000
      4     450000
      ...
      4335    409999
      4336    409999
      4337    110000
      4338    865000
      4339    225000
```

Name: selling_price, Length: 4340, dtype: int64

- **handle_unknown='ignore':** If the encoder sees a new category (e.g., a car brand not in the training set), it will simply represent it as all zeros in the one-hot encoded columns instead of crashing your app.
- **sparse_output=False:** (Optional but recommended for Streamlit) Returns a regular numpy array instead of a sparse matrix, which is easier to handle when passing data to the model for prediction.

```
[85]: num_features = X.select_dtypes(exclude="object").columns
      cat_features = X.select_dtypes(include="object").columns

      from sklearn.preprocessing import OneHotEncoder, StandardScaler
      from sklearn.compose import ColumnTransformer

      numerical_transformer = StandardScaler()
      hot_transformer = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

      preprocessor = ColumnTransformer ([
          ("OneHotEncoder", hot_transformer, cat_features),
          ("StandardScaler", numerical_transformer, num_features)
      ])

```

```
[86]: from sklearn.model_selection import train_test_split

      X_train, X_test, Y_train, Y_test = train_test_split(X, y, test_size=0.2,
      ↪train_size=0.8, random_state=2)

```

```
[87]: X_train = preprocessor.fit_transform(X_train)
      X_test = preprocessor.transform(X_test)

```

```
[88]: def evaluate_model(actual, predicted):
      mae = mean_absolute_error(actual, predicted)
      rmse = mean_squared_error(actual, predicted)
      r2_squared = r2_score(actual, predicted)
      return mae, rmse, r2_squared

```

```
[89]: model = LinearRegression(fit_intercept=True)
      model.fit(X_train, Y_train)

```

```
[89]: LinearRegression()

```

```
[90]: y_train_pred = model.predict(X_train)
      y_test_pred = model.predict(X_test)

      train_mae, train_mse, train_r2 = evaluate_model(Y_train, y_train_pred)
      test_mae, test_mse, test_r2 = evaluate_model(Y_test, y_test_pred)

      print("Linear Regression")
      print(f"Training: mae: {train_mae} rmse: {train_mse} r2: {train_r2}")
      print(f"Testing: mae: {test_mae} rmse: {test_mse} r2: {test_r2}")

```

Linear Regression

Training: mae: 40089.669930387354 rmse: 6340153822.45369 r2: 0.9811873639535998

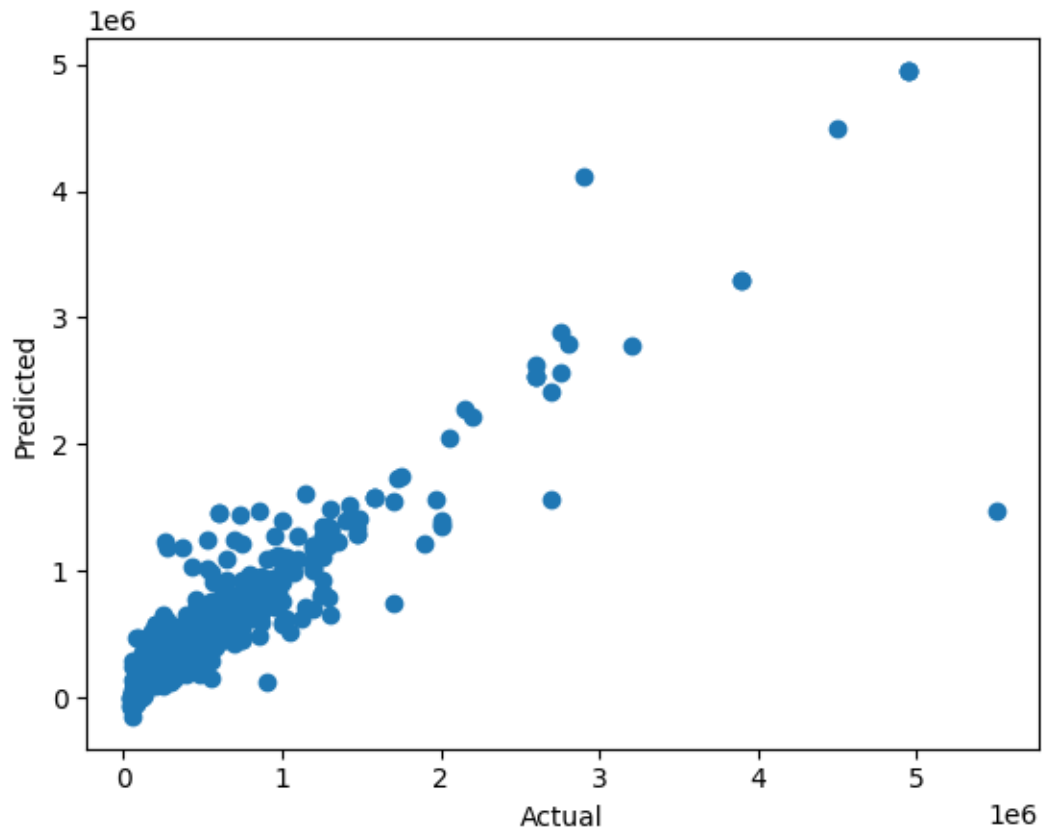
Testing: mae: 102598.25800404412 rmse: 49072580691.454315 r2: 0.8489896117972928

```
[91]: score = r2_score(Y_test, y_test_pred)*100  
print(f"Accuracy r2 score: {score}")
```

Accuracy r2 score: 84.89896117972928

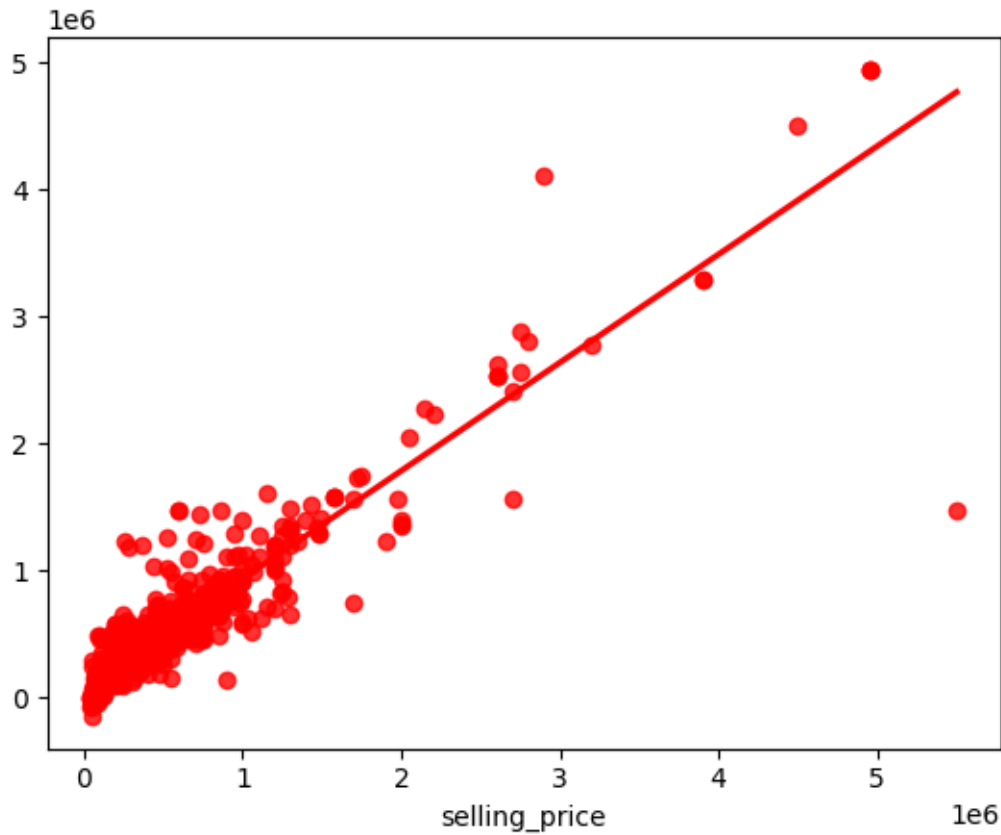
```
[92]: plt.scatter(Y_test, y_test_pred)  
plt.xlabel("Actual")  
plt.ylabel("Predicted")
```

```
[92]: Text(0, 0.5, 'Predicted')
```



```
[93]: sns.regplot(x=Y_test, y=y_test_pred, ci=None, color="red")
```

```
[93]: <Axes: xlabel='selling_price'>
```



```
[94]: pred_df = pd.DataFrame({"Actual":Y_test, "Predicted":y_test_pred, "Difference":
    ↪Y_test-y_test_pred})
pred_df.to_csv("actual_vs_predicted.csv", index=False)
pred_df
```

```
[94]:
```

	Actual	Predicted	Difference
1149	625000	8.579352e+05	-2.329352e+05
2245	520000	5.200000e+05	1.979060e-09
4261	115999	1.159990e+05	1.629815e-09
2865	2000000	1.366465e+06	6.335347e+05
3110	707000	7.145454e+05	-7.545432e+03
...	...	...	...
2168	100000	4.611362e+05	-3.611362e+05
3783	650000	9.227543e+05	-2.727543e+05
1683	430000	4.341147e+05	-4.114725e+03
2420	100000	3.099128e+05	-2.099128e+05
3354	300000	3.304982e+05	-3.049820e+04

```
[868 rows x 3 columns]
```

```
[95]: import joblib
      joblib.dump(model, "model.pkl")
      joblib.dump(preprocessor, "preprocessor.pkl")
```

```
[95]: ['preprocessor.pkl']
```